1️⃣ Trace & Dry Run Questions

Q1. Linear Search Trace

Given the array: `[3, 7, 1, 9, 5]`

- Search for `9` using **linear search**.
- Write the steps showing comparisons.
- How many comparisons were made?

Q2. Binary Search Trace (Iterative)

Given the sorted array: `[2, 4, 6, 8, 10, 12]`

- Search for `8` using **binary search**.
- Show the mid calculation and comparisons.
- How many steps did it take?

Q3. Binary Search Trace (Recursive)

Given the sorted array: `[1, 3, 5, 7, 9, 11]`

- Write the **recursive calls** when searching for `5`.
- Draw a small tree showing the calls and what each call returns.

2️⃣ Conceptual / Understanding Questions

Q4. Comparing Linear vs Binary Search

- Which search is better for large sorted arrays? Why?
- How many comparisons would linear search take in the worst case for an array of size `64`?
- How many for binary search?

Q5. Recursive Thinking

- Explain in words what recursion is doing in **binary search**.
- Why can't we pass a "subarray" in C easily?

3 Simple Coding / Pseudocode Questions

Q6. Write pseudocode

- Write **recursive binary search pseudocode** for an array of size `n`.
- Clearly indicate the base case and recursive step.

Q7. Write code snippet

- Write a function in C or Python that implements **linear search** and returns the index of the element or `-1` if not found.

4 Short Answer / Thought Questions

Q8. Thinking Questions

- What is the **time complexity** of linear and binary search?
- Why do we prefer binary search on large **sorted** arrays?
- Give one example where recursion makes a problem easier to think about than iteration.
- Suppose you have a **sorted array of unknown size**, but you can only read elements one by one. Could you still use binary search? Why or why not?